



K.N. Toosi University of Technology

OFFLINE JAVA GUIDE

Chapter 7

Lambdas & Streams

How to write concise behavior with lambda expressions and method references, and how to process collections declaratively using Java's Stream API.

COURSE

Advanced Programming

SEMESTER

Spring 2026

DEPARTMENT

Computer Engineering

v1.0

Table of Contents

7.1	Functional Interfaces	The road to lambdas
7.2	Lambda Expression Syntax	Forms and rules
7.3	Built-in Functional Interfaces	Predicate, Function, etc.
7.4	Method References	The :: operator
7.5	What is a Stream?	The pipeline model
7.6	Intermediate Operations	filter, map, sorted...
7.7	Terminal Operations	collect, reduce, count...
7.8	Collectors & Grouping	toList, groupingBy, joining
7.9	Optional	Avoiding null the safe way
7.10	Common Mistakes & Tips	Pitfalls every beginner hits

Before we can understand lambdas, we need to understand the problem they solve. In Java, you often need to pass **behavior** (a piece of code) into a method — not just data. Before Java 8, the only way to do this was with a verbose **anonymous class**.

The Problem — Passing Behavior

Suppose we want to sort a list of names by their *length*. The `sort` method needs to know *how* to compare two elements, so we pass it a `Comparator`:

```
1 List<String> names = new ArrayList<>(List.of("Mohammad", "Ali", "Sara", "Reza"));
2
3 // The OLD way — an anonymous class. Lots of boilerplate for one line of logic.
4 names.sort(new Comparator<String>() {
5     @Override
6     public int compare(String a, String b) {
7         return a.length() - b.length();
8     }
9 });
10
11 System.out.println(names);
```

JAVA

```
[Ali, Sara, Reza, Mohammad]
```

OUTPUT

Six lines, and only one of them — `a.length() - b.length()` — actually matters. Everything else is ceremony. This is exactly what lambdas eliminate:

```
1 // The NEW way — a lambda expression. Same behavior, one line.
2 names.sort((a, b) → a.length() - b.length());
```

JAVA

What is a Functional Interface?

A **functional interface** is an interface with **exactly one abstract method** (sometimes called a SAM — "Single Abstract Method"). Because there is only one method, Java can figure out which method your lambda implements. That is the whole trick that makes lambdas possible.

```
1 @FunctionalInterface
2 interface Greeting {
3     String sayHello(String name);    // exactly ONE abstract method
4 }
```

JAVA

The `@FunctionalInterface` annotation is optional but recommended: it tells the compiler to **enforce** that the interface has exactly one abstract method. If someone later adds a second abstract method, the code won't compile.

```
1 @FunctionalInterface
2 interface Greeting {
3     String sayHello(String name);
4     String sayBye(String name);    // COMPILE ERROR – two abstract methods!
5 }
```

Implementing It With a Lambda

```
1 @FunctionalInterface
2 interface Greeting {
3     String sayHello(String name);
4 }
5
6 public class Main {
7     public static void main(String[] args) {
8         // The lambda IS the implementation of sayHello
9         Greeting g = name → "Hello, " + name + "!";
10
11         System.out.println(g.sayHello("Sara"));
12         System.out.println(g.sayHello("Ali"));
13     }
14 }
```

Hello, Sara! Hello, Ali!

NOTE

You already used functional interfaces in Chapter 5 without knowing it. `Runnable`, `Comparator`, and `Comparable` are all functional interfaces — each has a single abstract method (`run`, `compare`, `compareTo`). A functional interface may also have `default` and `static` methods; only the **abstract** ones are counted.

TIP

Think of a lambda as a compact way to create an object that implements a one-method interface. The lambda is not "a function floating in space" — Java is still an object-oriented language, and behind the scenes the lambda becomes an instance of the target functional interface.

A lambda has two parts separated by the arrow token `→`: the **parameter list** on the left, and the **body** on the right.

```
1 (parameters) → expression
2 (parameters) → { statements; }
```

JAVA

The Different Forms

```
1 // No parameters - empty parentheses required
2 Runnable r = () → System.out.println("Running!");
3
4 // One parameter - parentheses are optional
5 Greeting g = name → "Hello, " + name;
6 Greeting g2 = (name) → "Hello, " + name; // same thing
7
8 // Multiple parameters - parentheses required
9 Comparator<String> byLen = (a, b) → a.length() - b.length();
10
11 // Explicit parameter types - allowed but usually unnecessary
12 Comparator<String> byLen2 = (String a, String b) → a.length() - b.length();
13
14 // Block body - needs braces AND an explicit return (if it returns a value)
15 Comparator<String> byLen3 = (a, b) → {
16     int diff = a.length() - b.length();
17     return diff;
18 };
```

JAVA

Expression Body vs Block Body

If the body is a **single expression**, you can omit the braces and the `return` keyword — the expression's value is returned automatically. If you need multiple statements, you must use braces `{ }` and an explicit `return`.

```
1 // Expression body - value is returned implicitly, no braces, no 'return'
2 Function<Integer, Integer> square = x → x * x;
3
4 // Block body - braces required, 'return' required
5 Function<Integer, Integer> squareVerbose = x → {
6     int result = x * x;
7     return result;
8 };
```

JAVA

MISTAKE

A very common beginner error: adding braces but forgetting `return`. `x → { x * x; }` does **not compile** if a value is expected — inside braces you must write `return x * x;`. Either use no braces (`x → x * x`) or use braces *with* return.

Capturing Variables (Closures)

A lambda can use variables from its surrounding scope. Those variables must be **final or effectively final** — meaning you never reassign them after initialization.

```
1 String prefix = "User: ";           // effectively final (never reassigned)
2 Greeting g = name → prefix + name; // lambda captures 'prefix'
3
4 System.out.println(g.sayHello("Ali")); // User: Ali
```

JAVA

```
1 String prefix = "User: ";
2 prefix = "Member: ";           // reassigned!
3 Greeting g = name → prefix + name; // COMPILER ERROR
4 // Variable used in lambda should be final or effectively final
```

JAVA

NOTE

Unlike anonymous classes, a lambda does **not** introduce a new scope for `this`. Inside a lambda, `this` refers to the enclosing object — not to the lambda itself. This is usually what you want, and it is one reason lambdas are less error-prone than anonymous classes.

7.3 Built-in Functional Interfaces

You rarely need to write your own functional interface. Java's `java.util.function` package ships with a set of general-purpose ones that cover almost every case. Learn these four first — the rest are variations.

INTERFACE	ABSTRACT METHOD	TAKES	RETURNS	MEANING
<code>Supplier<T></code>	<code>get()</code>	nothing	<code>T</code>	Produces a value
<code>Consumer<T></code>	<code>accept(T)</code>	<code>T</code>	nothing	Uses a value
<code>Function<T,R></code>	<code>apply(T)</code>	<code>T</code>	<code>R</code>	Transforms T into R
<code>Predicate<T></code>	<code>test(T)</code>	<code>T</code>	<code>boolean</code>	Tests a condition

`Predicate<T>` — a true/false test

```
1 Predicate<Integer> isEven = n → n % 2 == 0;
2 Predicate<String> isLong = s → s.length() > 5;
3
4 System.out.println(isEven.test(4));           // true
5 System.out.println(isEven.test(7));           // false
6 System.out.println(isLong.test("Mohammad")); // true
7
8 // Predicates combine with and(), or(), negate()
9 Predicate<Integer> isPositive = n → n > 0;
10 Predicate<Integer> isPositiveEven = isPositive.and(isEven);
11 System.out.println(isPositiveEven.test(6)); // true
12 System.out.println(isPositiveEven.test(-6)); // false
```

JAVA

```
true false true true false
```

OUTPUT

`Function<T,R>` — transform one value into another

```
1 Function<String, Integer> length = s → s.length();
2 Function<Integer, Integer> doubler = n → n * 2;
3
4 System.out.println(length.apply("Hello")); // 5
5
6 // andThen() chains functions: first length, then double
7 Function<String, Integer> lengthDoubled = length.andThen(doubler);
8 System.out.println(lengthDoubled.apply("Hello")); // 10
```

JAVA

```
5 10
```

OUTPUT

`Consumer<T>` — do something, return nothing

```
1 Consumer<String> printer = s → System.out.println(">> " + s);
2 printer.accept("Log message"); // >> Log message
3
4 // forEach on any collection takes a Consumer
5 List<String> names = List.of("Ali", "Sara", "Reza");
6 names.forEach(printer);
7 names.forEach(n → System.out.println("Name: " + n));
```

>> Log message >> Ali >> Sara >> Reza Name: Ali Name: Sara Name: Reza

Supplier<T> — produce a value on demand

```
1 Supplier<String> greeter = () → "Hello from a supplier";
2 Supplier<Double> randomValue = () → Math.random();
3
4 System.out.println(greeter.get());
5 System.out.println(randomValue.get());
```

More Variations You’ll Meet

INTERFACE	METHOD	PURPOSE
BiFunction<T,U,R>	apply(T,U)	Function of two arguments
BiConsumer<T,U>	accept(T,U)	Consumes two arguments
BiPredicate<T,U>	test(T,U)	Tests two arguments
UnaryOperator<T>	apply(T)	Function where input and output are the same type
BinaryOperator<T>	apply(T,T)	Combines two values of the same type into one

TIP

Don't memorize all of them. Ask two questions: **Does it take an input?** and **Does it return a value?** — Takes nothing, returns something → **Supplier** . Takes something, returns nothing → **Consumer** . Takes something, returns a boolean → **Predicate** . Takes something, returns something → **Function** .

7.4 Method References

When a lambda does nothing but call an existing method, you can replace it with a **method reference** using the `::` operator. It's shorter and reads like plain English.

```
1 // These two are equivalent:
2 Consumer<String> c1 = s → System.out.println(s); // lambda
3 Consumer<String> c2 = System.out::println;       // method reference
4
5 List<String> names = List.of("Ali", "Sara", "Reza");
6 names.forEach(System.out::println); // clean and readable
```

JAVA

Ali Sara Reza

OUTPUT

The Four Kinds of Method Reference

KIND	SYNTAX	LAMBDA EQUIVALENT
Static method	<code>Integer::parseInt</code>	<code>s → Integer.parseInt(s)</code>
Instance method of a <i>particular</i> object	<code>System.out::println</code>	<code>x → System.out.println(x)</code>
Instance method of an <i>arbitrary</i> object	<code>String::toUpperCase</code>	<code>s → s.toUpperCase()</code>
Constructor	<code>ArrayList::new</code>	<code>() → new ArrayList<>()</code>

Examples of Each

```
1 // 1. Static method reference
2 Function<String, Integer> parse = Integer::parseInt;
3 System.out.println(parse.apply("42") + 1); // 43
4
5 // 2. Instance method of a specific object
6 List<String> log = new ArrayList<>();
7 Consumer<String> adder = log::add; // always calls add() on THIS list
8 adder.accept("first");
9 adder.accept("second");
10 System.out.println(log); // [first, second]
11
12 // 3. Instance method of an arbitrary object of a type
13 Function<String, String> upper = String::toUpperCase;
14 System.out.println(upper.apply("hello")); // HELLO
15 // Here the String IS the argument – s.toUpperCase() is called on each input
16
17 // 4. Constructor reference
18 Supplier<ArrayList<String>> listMaker = ArrayList::new;
19 ArrayList<String> fresh = listMaker.get(); // new empty ArrayList
```

JAVA

NOTE

The trickiest one is #3 (arbitrary object). `String::toUpperCase` looks like it has no argument, but the *object the method is called on* becomes the lambda's parameter. So `String::toUpperCase` is `s → s.toUpperCase()`, and `String::compareTo` is `(a, b) → a.compareTo(b)`.

TIP

Write the lambda first. If your IDE (or you) notices it's just forwarding its arguments to a single method call, convert it to a method reference. If converting makes it *harder* to read, keep the lambda — clarity beats brevity.

A **Stream** is a sequence of elements that you process through a **pipeline** of operations. Instead of writing loops that describe *how* to iterate, you describe *what* you want done — filter these, transform those, collect the result.

The Old Way vs the Stream Way

Task: from a list of numbers, take the even ones, square them, and collect them into a new list.

```
1  List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6);
2
3  // IMPERATIVE - a manual loop describing every step
4  List<Integer> result = new ArrayList<>();
5  for (int n : numbers) {
6      if (n % 2 == 0) {
7          result.add(n * n);
8      }
9  }
10 System.out.println(result); // [4, 16, 36]
11
12 // DECLARATIVE - a stream pipeline describing the intent
13 List<Integer> result2 = numbers.stream()
14     .filter(n -> n % 2 == 0) // keep evens
15     .map(n -> n * n)         // square them
16     .toList();               // collect into a List
17 System.out.println(result2); // [4, 16, 36]
```

JAVA

```
[4, 16, 36] [4, 16, 36]
```

OUTPUT

Anatomy of a Stream Pipeline

Every stream pipeline has exactly three parts:

1. **Source** — where elements come from: `list.stream()`, `Stream.of(...)`, `Arrays.stream(arr)`.
2. **Intermediate operations** — transform the stream and return a new stream: `filter`, `map`, `sorted`. You can chain as many as you like.
3. **Terminal operation** — produces a result and ends the pipeline: `collect`, `toList`, `count`, `forEach`. Without one, nothing runs.

Three Key Properties

PROPERTY	WHAT IT MEANS
Lazy	Intermediate operations do nothing until a terminal operation is called. The pipeline is just a "recipe" until then.
Non-mutating	A stream never changes its source. <code>list.stream().sorted()</code> leaves the original list untouched.
Single-use	Once a terminal operation runs, the stream is consumed. You cannot reuse it — build a new one from the source.

WARNING

A stream can only be traversed **once**. Reusing a stream after its terminal operation throws `IllegalStateException: stream has already been operated upon or closed`. If you need the same pipeline twice, call `list.stream()` again to get a fresh stream.

```
1 Stream<Integer> s = List.of(1, 2, 3).stream();
2 s.forEach(System.out::println);    // OK - terminal operation runs
3 s.count();                        // RUNTIME ERROR - stream already consumed!
```

NOTE

Because streams are lazy, if you build a pipeline but never call a terminal operation, **none of the lambdas ever run**. A pipeline that ends at `.map(...)` with no `collect` / `forEach` / `count` does absolutely nothing.

Intermediate operations return a **new stream**, so they can be chained. Here are the ones you'll use constantly. We'll use this small data set throughout:

```
1 List<String> words = List.of("apple", "banana", "avocado", "cherry", "apricot", "kiwi");
```

JAVA

filter — keep elements that match a Predicate

```
1 List<String> aWords = words.stream()
2   .filter(w → w.startsWith("a"))
3   .toList();
4 System.out.println(aWords);    // [apple, avocado, apricot]
```

JAVA

map — transform each element

```
1 List<Integer> lengths = words.stream()
2   .map(String::length)
3   .toList();
4 System.out.println(lengths);    // [5, 6, 7, 6, 7, 4]
5
6 List<String> upper = words.stream()
7   .map(String::toUpperCase)
8   .toList();
9 System.out.println(upper);      // [APPLE, BANANA, AVOCADO, ...]
```

JAVA

sorted — order the elements

```
1 // Natural order (alphabetical for Strings)
2 List<String> sorted = words.stream().sorted().toList();
3 System.out.println(sorted);
4 // [apple, apricot, avocado, banana, cherry, kiwi]
5
6 // Custom order via a Comparator — here, by length
7 List<String> byLength = words.stream()
8   .sorted(Comparator.comparingInt(String::length))
9   .toList();
10 System.out.println(byLength);
11 // [kiwi, apple, banana, cherry, avocado, apricot]
12
13 // Reversed
14 List<String> byLengthDesc = words.stream()
15   .sorted(Comparator.comparingInt(String::length).reversed())
16   .toList();
```

JAVA

distinct, limit, skip

```
1 List<Integer> nums = List.of(1, 2, 2, 3, 3, 3, 4, 5);
2
3 List<Integer> unique = nums.stream().distinct().toList();
4 System.out.println(unique);    // [1, 2, 3, 4, 5]
5
6 List<Integer> firstThree = nums.stream().distinct().limit(3).toList();
7 System.out.println(firstThree); // [1, 2, 3]
8
9 List<Integer> afterTwo = nums.stream().distinct().skip(2).toList();
10 System.out.println(afterTwo);  // [3, 4, 5]
```

[1, 2, 3, 4, 5] [1, 2, 3] [3, 4, 5]

peek — look without changing (debugging)

```
1 List<String> result = words.stream()
2     .filter(w → w.length() > 5)
3     .peek(w → System.out.println("passed filter: " + w))
4     .map(String::toUpperCase)
5     .toList();
```

Chaining Them Together

The power comes from combining operations. Read top to bottom like a sentence:

```
1 List<String> result = words.stream()
2     .filter(w → w.length() > 4)    // longer than 4 letters
3     .map(String::toUpperCase)      // upper-cased
4     .sorted()                      // alphabetical
5     .limit(3)                      // first 3 only
6     .toList();
7 System.out.println(result);        // [APPLE, APRICOT, AVOCADO]
```

OPERATION	TAKES	EFFECT
<code>filter(Predicate)</code>	a test	Keeps only elements that pass
<code>map(Function)</code>	a transform	Replaces each element with the result
<code>sorted()</code> / <code>sorted(Comparator)</code>	optional order	Sorts the elements
<code>distinct()</code>	—	Removes duplicates (uses <code>equals</code>)
<code>limit(n)</code>	a count	Keeps only the first <code>n</code>
<code>skip(n)</code>	a count	Discards the first <code>n</code>

Order matters for performance. Put `filter` **before** `map` and `sorted` when you can — filtering first means fewer elements to transform and sort. A `limit` early can also let a lazy stream stop work sooner.

A **terminal operation** triggers the pipeline and produces a result — a value, a collection, or a side effect. After it runs, the stream is done.

forEach — perform an action on each element

```
1 List.of("Ali", "Sara", "Reza").stream()
2     .forEach(name → System.out.println("Hi " + name));
```

JAVA

count — how many elements

```
1 long count = words.stream()
2     .filter(w → w.startsWith("a"))
3     .count();
4 System.out.println(count);    // 3
```

JAVA

collect / toList — gather into a collection

```
1 // toList() (Java 16+) – returns an unmodifiable List, the quick option
2 List<String> list = words.stream().filter(w → w.length() > 5).toList();
3
4 // collect(Collectors.toList()) – the classic, fully general form
5 List<String> list2 = words.stream()
6     .filter(w → w.length() > 5)
7     .collect(Collectors.toList());
8
9 // collect into a Set to drop duplicates
10 Set<Integer> lengths = words.stream()
11     .map(String::length)
12     .collect(Collectors.toSet());
```

JAVA

reduce — combine all elements into one

reduce repeatedly combines elements using a **BinaryOperator** until a single value remains — a sum, a product, a concatenation, a maximum.

```

1 List<Integer> nums = List.of(1, 2, 3, 4, 5);
2
3 // With an identity (starting value) – never empty, returns int
4 int sum = nums.stream().reduce(0, (a, b) → a + b);
5 System.out.println(sum);    // 15
6
7 int product = nums.stream().reduce(1, (a, b) → a * b);
8 System.out.println(product); // 120
9
10 // Without identity – result may be absent, so it returns Optional
11 Optional<Integer> max = nums.stream().reduce((a, b) → a > b ? a : b);
12 System.out.println(max.get()); // 5

```

15 120 5

OUTPUT

Matching & Finding

```

1 List<Integer> nums = List.of(2, 4, 6, 8);
2
3 boolean allEven = nums.stream().allMatch(n → n % 2 == 0); // true
4 boolean anyOver5 = nums.stream().anyMatch(n → n > 5);      // true
5 boolean noneNeg = nums.stream().noneMatch(n → n < 0);      // true
6
7 // findFirst / findAny return Optional (the stream might be empty)
8 Optional<Integer> first = nums.stream().filter(n → n > 3).findFirst();
9 System.out.println(first.get()); // 4

```

4

OUTPUT

Numeric Aggregates with IntStream

For numbers, converting to a specialized `IntStream` (via `mapToInt`) unlocks convenient math methods:

```

1 List<String> words = List.of("apple", "banana", "kiwi");
2
3 int totalChars = words.stream().mapToInt(String::length).sum();
4 System.out.println(totalChars); // 15
5
6 double avgLength = words.stream().mapToInt(String::length).average().orElse(0);
7 System.out.println(avgLength); // 5.0
8
9 int maxLength = words.stream().mapToInt(String::length).max().orElse(0);
10 System.out.println(maxLength); // 6

```

15 5.0 6

OUTPUT

The `collect` terminal operation is the most powerful one. Paired with the `Collectors` utility class, it can gather stream elements into lists, sets, maps, joined strings, groups, and statistics. We'll use this small `Person` record throughout:

```
1 record Person(String name, String city, int age) {}
2
3 List<Person> people = List.of(
4     new Person("Ali", "Tehran", 22),
5     new Person("Sara", "Shiraz", 25),
6     new Person("Reza", "Tehran", 30),
7     new Person("Mina", "Shiraz", 19),
8     new Person("Omid", "Tabriz", 22)
9 );
```

JAVA

toList / toSet / toMap

```
1 // Collect names into a List
2 List<String> names = people.stream().map(Person::name).collect(Collectors.toList());
3
4 // Build a Map from name to age
5 Map<String, Integer> ageByName = people.stream()
6     .collect(Collectors.toMap(Person::name, Person::age));
7 System.out.println(ageByName);
8 // {Ali=22, Sara=25, Reza=30, Mina=19, Omid=22}
```

JAVA

joining — build one String

```
1 String allNames = people.stream()
2     .map(Person::name)
3     .collect(Collectors.joining(", "));
4 System.out.println(allNames); // Ali, Sara, Reza, Mina, Omid
5
6 // With prefix and suffix
7 String bracketed = people.stream()
8     .map(Person::name)
9     .collect(Collectors.joining(", ", "[", "]"));
10 System.out.println(bracketed); // [Ali, Sara, Reza, Mina, Omid]
```

JAVA

```
Ali, Sara, Reza, Mina, Omid [Ali, Sara, Reza, Mina, Omid]
```

OUTPUT

groupingBy — the workhorse

`groupingBy` splits elements into a `Map` where the key is some property and the value is a list of the matching elements. This replaces a whole nest of loops and maps.

```

1 Map<String, List<Person>> byCity = people.stream()
2   .collect(Collectors.groupingBy(Person::city));
3
4 System.out.println(byCity.get("Tehran"));
5 // [Person[name=Ali, city=Tehran, age=22], Person[name=Reza, city=Tehran, age=30]]

```

groupingBy With a Downstream Collector

The second argument tells `groupingBy` what to do with each group instead of just listing them — count them, sum them, map them.

```

1 // Count people per city
2 Map<String, Long> countByCity = people.stream()
3   .collect(Collectors.groupingBy(Person::city, Collectors.counting()));
4 System.out.println(countByCity); // {Tehran=2, Shiraz=2, Tabriz=1}
5
6 // Average age per city
7 Map<String, Double> avgAgeByCity = people.stream()
8   .collect(Collectors.groupingBy(Person::city,
9     Collectors.averagingInt(Person::age)));
10 System.out.println(avgAgeByCity); // {Tehran=26.0, Shiraz=22.0, Tabriz=22.0}
11
12 // Just the names in each city
13 Map<String, List<String>> namesByCity = people.stream()
14   .collect(Collectors.groupingBy(Person::city,
15     Collectors.mapping(Person::name, Collectors.toList())));
16 System.out.println(namesByCity);
17 // {Tehran=[Ali, Reza], Shiraz=[Sara, Mina], Tabriz=[Omid]}

```

{Tabriz=1, Tehran=2, Shiraz=2} {Tabriz=22.0, Tehran=26.0, Shiraz=22.0} {Tabriz=[Omid], Tehran=[Ali, Reza], Shiraz=[Sara, Mina]}

partitioningBy — split into true/false

```

1 // Divide into two groups by a Predicate
2 Map<Boolean, List<Person>> adults = people.stream()
3   .collect(Collectors.partitioningBy(p -> p.age() ≥ 21));
4
5 System.out.println("Adults: " + adults.get(true));
6 System.out.println("Minors: " + adults.get(false));

```

A `HashMap` has no guaranteed order, so the key order you see when printing a grouped map may differ from the insertion order of your data. That's expected. If you need a predictable order, collect into a `TreeMap` or sort the entries afterward.

Many stream operations (`findFirst` , `reduce` , `max`) can't guarantee a result — the stream might be empty. Instead of returning `null` (and risking a `NullPointerException`), they return an `Optional<T>` : a box that either *contains* a value or is explicitly *empty*.

Creating an Optional

```
1 Optional<String> present = Optional.of("hello");    // holds a value
2 Optional<String> empty   = Optional.empty();       // holds nothing
3 Optional<String> maybe   = Optional.ofNullable(getName()); // value or empty if null
```

JAVA

Reading the Value Safely

```
1 List<Integer> nums = List.of(3, 8, 1, 9, 4);
2 Optional<Integer> max = nums.stream().max(Comparator.naturalOrder());
3
4 // Check first, then get
5 if (max.isPresent()) {
6     System.out.println("Max: " + max.get());
7 }
8
9 // Provide a default with orElse
10 int biggest = max.orElse(0);
11 System.out.println(biggest);    // 9
12
13 // Run code only if present
14 max.ifPresent(m → System.out.println("Found " + m));
```

JAVA

Max: 9 9 Found 9

OUTPUT

orElse, orElseGet, orElseThrow

```
1 Optional<String> name = findUserName();    // may be empty
2
3 // 1. A plain fallback value
4 String a = name.orElse("Guest");
5
6 // 2. A fallback computed lazily (only if empty) — good for expensive defaults
7 String b = name.orElseGet(() → loadDefaultName());
8
9 // 3. Throw if absent — when empty truly is an error
10 String c = name.orElseThrow(() → new NoSuchElementException("No user!"));
```

JAVA

Transforming Inside an Optional

`map` lets you work on the value *only if present*, without unpacking it manually:

```
1 Optional<String> name = Optional.of("ali");
2
3 Optional<Integer> length = name.map(String::length); // Optional[3]
4 String upper = name.map(String::toUpperCase).orElse("UNKNOWN");
5 System.out.println(upper); // ALI
```

MISTAKE

Do not call `.get()` without checking first. On an empty `Optional` it throws `NoSuchElementException: No value present` — the exact crash `Optional` was designed to prevent. Prefer `orElse`, `orElseGet`, `ifPresent`, or `orElseThrow` with a clear message.

TIP

Use `Optional` as a **return type** for methods that may have no answer (e.g. "find a user by id"). Don't use it for fields, method parameters, or collections — an empty `List` already expresses "nothing here" without the extra wrapper.

1. Reusing a Consumed Stream

```
1 // WRONG - a stream is single-use
2 Stream<Integer> s = List.of(1, 2, 3).stream();
3 long c = s.count();
4 s.forEach(System.out::println); // IllegalStateException!
5
6 // CORRECT - create a fresh stream each time
7 List<Integer> data = List.of(1, 2, 3);
8 long c2 = data.stream().count();
9 data.stream().forEach(System.out::println);
```

JAVA

2. Forgetting the Terminal Operation

```
1 // WRONG - no terminal op, so NOTHING happens (streams are lazy)
2 List.of("a", "b", "c").stream()
3     .map(String::toUpperCase)
4     .peek(System.out::println); // never prints!
5
6 // CORRECT - add a terminal operation to trigger the pipeline
7 List<String> result = List.of("a", "b", "c").stream()
8     .map(String::toUpperCase)
9     .toList();
```

JAVA

3. Mutating State From Inside a Stream

```
1 // BAD - side effects in map/filter; fragile and breaks in parallel
2 List<Integer> result = new ArrayList<>();
3 List.of(1, 2, 3).stream().forEach(n → result.add(n * 2)); // avoid
4
5 // GOOD - let the stream build the result declaratively
6 List<Integer> result2 = List.of(1, 2, 3).stream()
7     .map(n → n * 2)
8     .toList();
```

JAVA

4. Trying to Modify the Captured Variable

```
1 // WRONG - captured variables must be effectively final
2 int total = 0;
3 List.of(1, 2, 3).forEach(n → total += n); // COMPILER ERROR
4
5 // CORRECT - use a reduction or a numeric stream
6 int total2 = List.of(1, 2, 3).stream().mapToInt(Integer::intValue).sum();
```

JAVA

5. Overusing Streams for Simple Loops

```

1 // OVERKILL – a stream to print three lines is harder to read than a loop
2 List.of("a", "b").stream().forEach(System.out::println);
3
4 // Sometimes the plain loop is clearer – that's fine!
5 for (String s : List.of("a", "b")) {
6     System.out.println(s);
7 }

```

WARNING

Streams shine for **transforming and aggregating data** (filter → map → collect). They are *not* a replacement for every loop. If your loop mutates external state, breaks early, or is just simpler as a loop — use a loop. Readability first.

6. Autoboxing Overhead in Numeric Pipelines

```

1 // SLOWER – boxes every int into an Integer object
2 int sum = List.of(1, 2, 3).stream().reduce(0, Integer::sum);
3
4 // FASTER – mapToInt uses a primitive IntStream, no boxing
5 int sum2 = List.of(1, 2, 3).stream().mapToInt(Integer::intValue).sum();

```

QUICK REFERENCE – LAMBDA & STREAM DECISIONS

- › A lambda only works where a **functional interface** is expected (one abstract method)
- › Use a **method reference** when the lambda just forwards to one existing method
- › Every pipeline = **source** → **intermediate ops** → **one terminal op**
- › Intermediate ops are **lazy**; nothing runs until the terminal op
- › A stream is **single-use** — get a fresh one from the source each time
- › Prefer `filter` before `map` / `sorted` to do less work
- › Use `Optional` to return "maybe a value"; never call `.get()` unchecked
- › Don't force streams onto every loop — use them to **transform and aggregate** data

QUICK REFERENCE – FUNCTIONAL INTERFACE CHEAT SHEET

- › `Supplier<T>` — `() → T` — takes nothing, produces a value
- › `Consumer<T>` — `T → void` — takes a value, returns nothing
- › `Predicate<T>` — `T → boolean` — tests a condition
- › `Function<T,R>` — `T → R` — transforms T into R
- › `UnaryOperator<T>` — `T → T` — same input and output type
- › `BinaryOperator<T>` — `(T,T) → T` — combine two into one

This document was compiled from the following resources. All content was adapted and restructured for educational use.

- [1] **Oracle — Java Tutorials: Lambda Expressions**
Functional interfaces, lambda syntax, method references, variable capture
<https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html>
- [2] **Oracle — Java Tutorials: Aggregate Operations**
Stream pipelines, intermediate & terminal operations, reduction
<https://docs.oracle.com/javase/tutorial/collections/streams/>
- [3] **Programiz — Java Lambda Expressions**
Syntax forms, functional interfaces, practical examples
<https://www.programiz.com/java-programming/lambda-expression>
- [4] **GeeksforGeeks — Stream In Java**
Stream creation, operations, collectors, grouping, and common pitfalls
<https://www.geeksforgeeks.org/stream-in-java/>
- [5] **Java Platform SE — `java.util.function` & `java.util.stream` — API documentation**
Predicate, Function, Consumer, Supplier, Stream, Collectors, Optional
<https://docs.oracle.com/en/java/javase/17/docs/api/>

CREATED BY

Sepehr Ghardashi

sepehr.ghardashi@gmail.com